

课程讲义

[CS 153] Cursor 的基础设施与 AI 编程 — CTO Sualeh Asif

基于 Stanford CS 153 课程内容整理

2025



课程	Stanford CS 153: Infrastructure at Scale
嘉宾	Sualeh Asif (Cursor CTO & Co-Founder)
频道	Stanford CS 153
发布日期	2025-03-04
时长	48:37
视频链接	https://www.youtube.com/watch?v=4jDQI9P9UIw

本讲义基于课堂对话录音整理，技术术语保留英文原文。
项目地址：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：Cursor 的规模与增长	3
1.1 本章小结	3
2 基础设施三大支柱	3
2.1 索引系统	3
2.2 模型推理	4
2.3 数据流基础设施	4
2.4 产品层：Apply 模型与 UX 优化	4
2.5 本章小结	4
3 架构设计：单体服务与故障隔离	4
3.1 单体架构的选择	4
3.2 爆炸半径控制	5
3.3 代码简洁性的强制要求	5
3.4 本章小结	5
4 索引系统：Merkle Tree 与数据库演进	5
4.1 Merkle Tree 同步机制	5
4.2 数据库选型的惨痛教训	6
4.2.1 YugaByte 的失败	6
4.2.2 PostgreSQL 的 MVCC 陷阱	6
4.3 本章小结	6
5 重大事故与应急响应	7
5.1 事故一：索引系统连环故障	7
5.1.1 故障链	7
5.1.2 排查过程	7
5.2 事故二：PostgreSQL 存储危机	7
5.2.1 故障背景	7
5.2.2 War Room 应急	8
5.2.3 外部求助	8
5.2.4 戏剧性的结局	8
5.3 Cold Start 问题	8
5.4 本章小结	9
6 数据库技术趋势：Object Storage 革命	9
6.1 Object Storage 的优势	9
6.2 代表性项目	9
6.3 本章小结	9
7 模型推理与多 Provider 策略	10
7.1 全球 GPU 部署	10
7.2 Frontier Model 供应商管理	10

7.3 本章小结	11
8 安全、定价与反滥用	11
8.1 代码安全	11
8.2 定价策略	11
8.3 反滥用工程	11
8.4 本章小结	11
9 AI 对软件工程的影响	12
9.1 竞争与差异化	12
9.2 CS 教育是否过时?	12
9.3 IDE 的未来	12
9.4 AI 辅助的事故响应	12
9.5 本章小结	12
10 总结与延伸	13
10.1 拓展阅读	13

1 引言：Cursor 的规模与增长

本次讲座是 Stanford CS 153 (Infrastructure at Scale) 第五周的嘉宾分享。Cursor 的 CTO 兼联合创始人 Sualeh Asif 受邀深入讲述 Cursor 产品背后的基础设施架构、重大事故处理经验，以及 AI 编程工具面临的独特工程挑战。课堂采用对话式问答形式，由主持人 Andre 提问，学生在 Discord 中实时提交问题。

Cursor 的规模数据

- 过去一年增长超过 **100 倍**，部分指标增长更快
- 自研模型（custom models）每天处理约 **1 亿次模型调用**（model calls）
- 索引系统每天处理约 **10 亿份文档**，公司生命周期内已索引数千亿份文档
- 占 Frontier Model 流量的相当大比例，是多个模型供应商的最大客户之一

Asif 指出，Cursor 的增长速度带来了极大的基础设施压力——这不仅仅是简单的“加服务器”问题，而是涉及全球分布式系统、数据库选型、故障恢复等一系列深层次的工程挑战。

1.1 本章小结

Cursor 在过去一年经历了百倍级别的增长，日均处理 1 亿次自研模型调用和 10 亿份文档索引。这种增长速度意味着基础设施团队必须持续应对前所未有的规模化挑战。

2 基础设施三大支柱

Asif 将 Cursor 的基础设施分为三个核心部分：索引系统（Indexing）、模型推理（Models）和数据流处理（Streaming Infrastructure）。

2.1 索引系统

索引系统是 Cursor 最为人知的基础设施之一。它包含多个子系统：

- **检索系统（Retrieval Systems）**：支撑代码库搜索，当用户向 Agent 提问时，系统从代码库中检索相关上下文
- **Git 索引**：深度理解用户的代码仓库历史，包括处理像 Instagram 这样规模的大型仓库
- **自动索引**：用户打开编辑器时自动触发，不需要任何手动操作

为什么自动索引至关重要

Asif 强调：如果给用户添加过多按钮和操作步骤来索引代码库，没有人会去用它。唯一的办法是让索引完全自动化——用户打开编辑器的那一刻就开始索引。除非代码库大到了 Instagram 的级别，Cursor 会毫不犹豫地索引所有内容，并承担全部成本。

2.2 模型推理

模型推理是 Cursor 最计算密集的部分，分为两个层面：

1. **自研模型推理**：Autocomplete 模型在用户每次击键时触发，任意时刻并发约 **20,000 次/秒** 模型调用。运行在约 1,000–2,000 张 H100 GPU 的集群上，分布在美国东海岸（Virginia）、西海岸（Phoenix）、伦敦和东京等地。
2. **Frontier Model 推理**：依赖 Anthropic、OpenAI、Google Cloud 等第三方模型供应商，用于 Chat、Composer 等高级功能。

Autocomplete 的极端需求

每次用户按键都会触发 Autocomplete 模型调用，每次调用处理数万 tokens 的上下文。按 Asif 的估算，每次击键可能涉及约 1000 亿次浮点运算（FLOPs）。这意味着 Autocomplete 在 input token 数量上远远超过 Frontier Model 的使用量。

2.3 数据流基础设施

第三个支柱是后台的数据流处理系统，用于：

- 存储和处理用户使用数据
- 支撑模型训练和评估
- 日常改进产品体验

这不是用户直接交互的实时系统，而是一个后台持续运行的“改进引擎”。

2.4 产品层：Apply 模型与 UX 优化

除了三大核心支柱，Asif 还提到了产品层的工程技巧。Apply 模型需要处理 100,000–200,000 tokens 的上下文，但必须让用户感觉像是简单的复制粘贴——速度快到感觉不到模型在工作。这涉及大量推理侧的优化技巧。

2.5 本章小结

Cursor 的基础设施由索引、模型推理和数据流三大支柱构成。索引系统追求完全自动化；模型推理分为自研（高频低延迟的 Autocomplete）和第三方 Frontier Model 两部分；数据流系统在后台持续驱动产品改进。

3 架构设计：单体服务与故障隔离

3.1 单体架构的选择

Asif 指出，Cursor 采用的是**单体架构 (Monolithic Architecture)**——所有服务打包成一个大型 monolith 进行部署。这在快速迭代的初创公司中是常见的选择。

单体 vs 微服务的权衡

尽管微服务架构在大型企业中流行，但对于快速成长的初创公司，单体架构有其显著优势：部署简单、开发速度快、调试容易。Cursor 选择在单体架构内部做隔离，而非过早拆分微服务。

3.2 爆炸半径控制

随着规模增长，**爆炸半径 (Blast Radius)** 控制变得至关重要。Asif 分享了一个早期教训 (2023 年 6-7 月)：

一个无限循环就能搞垮全部服务

早期 Cursor 所有功能运行在同一台服务器上。有人写了一个死循环 (infinite loop)，导致 Chat 功能挂掉——更糟糕的是，连登录服务也一起挂了。核心教训：**不能让实验性代码的 bug 影响到关键服务** (如认证、支付等)。

解决方案是对服务器进行**分区隔离 (Compartmentalization)**：将关键服务放在更安全、更受保护的区域，与实验性代码隔开。

3.3 代码简洁性的强制要求

Asif 强调，随着规模增长，团队对服务端代码的复杂度有了**严格的限制**：如果代码太复杂、无法被人理解，就不应该上线运行。这已经成为 Cursor 基础设施管理的重要原则。

3.4 本章小结

Cursor 采用单体架构并在内部做隔离来控制爆炸半径。早期的无限循环事故促使团队将关键服务与实验性代码严格分离。服务端代码复杂度受到严格管控，确保可理解性和可运维性。

4 索引系统：Merkle Tree 与数据库演进

4.1 Merkle Tree 同步机制

Cursor 的索引系统使用 **Merkle Tree** (默克尔树) 来高效同步客户端与服务端的文件状态。

Merkle Tree 文件同步原理

1. 每个文件被 hash，每个文件夹的 hash 等于其所有子节点 hash 的组合，一直到根节点
2. 当用户重新打开编辑器 (如 git checkout 到一年前的版本)，客户端计算本地 Merkle Tree，与服务端对比
3. 从根节点开始：如果 hash 不同，至少有一个子文件夹发生了变化
4. 逐层向下遍历，精确定位变化的文件
5. 只上传发生变化的文件进行重新索引

这种机制使得 Cursor 能够高效处理大规模代码库的增量更新，而不需要每次都重新索引整个代码库。

4.2 数据库选型的惨痛教训

索引系统的数据库选型经历了一段曲折的历程：

分布式数据库领域的“三巨头”

Asif 介绍了数据库世界中几个标志性的系统：

- **Google Spanner**：以全球级别的分布式事务能力著称
- **FoundationDB**：Apple 内部广泛使用的分布式 KV 存储
- **Redis** 等 KV 存储：简单高效的键值数据库

Asif 认为，如果你有这三种类型的数据库加上一个分析引擎，基本可以应对所有场景。

4.2.1 YugaByte 的失败

作为 Google Spanner 的开源后代，YugaByte 在理论上具备无限可扩展性。Asif 选择它正是看中了这一点。然而现实是：

不要选择过于复杂的数据库

YugaByte 在 Cursor 的场景下根本跑不起来。花了大量资金试图缩减节点，但系统就是不稳定。其全球分布式事务的共识协议（consensus protocol）在大量长事务场景下表现极差。Asif 的教训：**先用 Postgres，不要用花哨的分布式数据库**。迁移到 AWS RDS (Postgres) 后，问题迎刃而解。

4.2.2 PostgreSQL 的 MVCC 陷阱

迁移到 RDS 后，索引系统平稳运行了八个月。然而，22 TB 数据量最终引发了一场严重事故。

PostgreSQL Update

PostgreSQL 与 MySQL 在 Update 操作上有根本区别：

- **MySQL**：直接在磁盘上原地修改记录
- **PostgreSQL**：Update 实际上是 Delete（标记 tombstone）+ Insert（写入新记录），旧记录的磁盘空间不会立即回收

对于 Cursor 这样“用户每次击键都触发 Update”的工作负载，会产生**海量的死元组（dead tuples）**，导致存储空间快速膨胀。

PostgreSQL 依赖后台的 **VACUUM** 进程来清理死元组、回收空间，以及 **Anti-Wraparound Vacuum** 来处理事务 ID 回绕问题。当这两个进程同时出现异常时，数据库就会陷入恶性循环。

4.3 本章小结

Cursor 使用 Merkle Tree 实现高效的客户端-服务端文件同步。数据库选型经历了 YugaByte 到 PostgreSQL 的迁移。PostgreSQL 的 MVCC 机制（Update = Delete + Insert）在高频更新场景下会产生严重的存储膨胀问题，需要特别关注 VACUUM 配置。

5 重大事故与应急响应

Asif 分享了两次重大事故 (Sev) 的详细经过, 展现了规模化服务中故障处理的残酷现实。

5.1 事故一: 索引系统连环故障

5.1.1 故障链

这次事故涉及多个系统的级联故障:

1. **缓存层 Bug**: 使用 AWS DynamoDB 作为缓存, 但存在一个隐蔽的 bug——超过一定大小的文件无法被缓存到 DynamoDB (DynamoDB 有 item 大小限制)
2. **缓存未命中**: 大文件每次都 miss cache, 直接打到 Embedding 模型, 导致模型过载
3. **队列竞态条件**: 完成 chunking 和 embedding 后, 需要更新 Merkle Tree 的上层 hash, 这个过程在队列上存在竞态条件 (race condition), 导致大文件的提交永远无法完成
4. **无限重试循环**: 客户端发现大文件未提交, 重新发送请求 → 缓存再次 miss → 再次打 Embedding 模型 → 再次提交失败 → 无限循环

全局错误率”正常”的假象

最危险的地方在于: 全局的错误率指标看起来完全正常! 只有那些超大文件 (60,000 行以上) 在不断重试。这意味着传统的监控指标无法发现问题, 需要细粒度地监控缓存命中率等更具体的指标。

5.1.2 排查过程

团队首先注意到缓存命中率 (cache hit rate) 异常, 然后发现文件未能成功提交, 最终定位到 DynamoDB 的大小限制和队列上的竞态条件。Asif 坦言, 分布式系统中的竞态条件即使你知道它存在, 要真正找到它也是**极其困难的**。

5.2 事故二: PostgreSQL 存储危机

5.2.1 故障背景

索引系统迁移到 RDS 后平稳运行了八个月, 团队甚至忘记了监控 dashboard 的存在。直到某天 Asif 被大量告警页面唤醒:

- RDS 数据量达到 22 TB (上限 64 TB)
- 看似还有空间, 但实际上已经岌岌可危
- VACUUM 和 Anti-Wraparound Vacuum 同时失控
- 数据库表现为”引擎抖动”: 刚恢复就再次崩溃, 反复循环

5.2.2 War Room 应急

Asif 描述了一个经典的 War Room 场景：

并行应急策略

团队同时派出多人执行不同方案：

1. **A 组**：删除数据库中所有 foreign key 约束，降低数据库负载
2. **B 组**：尝试删除最大的 20 TB 表
3. **C 组**：手动在 PostgreSQL 控制台中清理事务，维持数据库存活
4. **D 组**：尝试创建 replica 并迁移
5. **E 组（联合创始人 Arvid）**：重写整个工作负载，将最大的表迁移到 Object Storage

5.2.3 外部求助

- 联系了一位朋友 Martin（前 GitHub VP of Infrastructure），他推荐了 RDS 专家
- 联系 AWS Support——完全无法提供有用帮助
- 最终与 RDS 的底层架构师通话，他们的回答是：“你们确实陷入了大麻烦”

事故中的两种人

Asif 观察到，面对重大事故有两种人：一种人会变得焦虑（get shitty），另一种人反而会“苏醒”（come alive）。他的联合创始人 Arvid 属于后者——平时不太关注日常运维，但一到重大事故就变得异常兴奋和高效。这种人在危机时刻是无价的。

5.2.4 戏剧性的结局

最终，负责重写工作负载的 Arvid 竟然比所有其他方案更快地完成了任务——他在事故进行中（约 10 小时内）将最大的数据表迁移到了 Object Storage，然后新系统上线，流量切换过去，危机解除。

在生产环境运行未经测试的代码

Asif 坦言，这段代码是在极端压力下几小时内写出的，没有测试，没有代码审查，直接部署到处理最高流量的生产服务上。这在正常情况下是不可接受的，但当数据库已经完全不可用时，“反正什么都不能工作，不如放手一搏”。

5.3 Cold Start 问题

Asif 特别提到了服务恢复中的 **Cold Start Problem**（冷启动问题）：

当所有节点都宕机后（如处理 100K requests/s 的集群），你不能直接把所有节点同时拉起来：

- 如果集群有 1,000 个节点，先起来 10 个
- 所有积压的请求会涌向这 10 个节点，把它们直接压垮

- 10 个节点还没来得及变健康，就已经被”杀死”了
- 你需要先**切断大部分流量**，然后逐步放量

WhatsApp 的做法是使用前缀分组——如果整个系统崩溃，只先恢复特定前缀的用户群，而不是一次性服务所有人。Cursor 的做法是对用户设置优先级，或者对所有用户统一限流。

5.4 本章小结

重大事故的处理需要并行执行多个方案、快速做出决策，以及团队中有能在危机时刻高效工作的关键人物。冷启动问题是服务恢复中的一个核心挑战。监控指标的粒度决定了你能否及时发现问题——全局错误率可能掩盖局部严重故障。

6 数据库技术趋势：Object Storage 革命

事故的解决方案——将数据迁移到 Object Storage——背后有一个更深层的技术趋势。

6.1 Object Storage 的优势

Asif 认为，将数据库构建在 Object Storage（如 S3、R2、Azure Blob）之上是近年来最重要的数据库趋势之一：

Object Storage 的三大核心优势

- **极致可扩展性**：已被优化到极致，几乎没有容量上限
- **极高可靠性**：世界上没有比 Object Storage 更可靠的存储系统
- **消除存储层复杂性**：数据库中最难的部分是存储层，Object Storage 将这个问题完全托管

6.2 代表性项目

- **WarpStream**（被 Confluent 收购）：将 Kafka 重写到 Blob Storage 之上。Asif 特别提到，Kafka 在基础设施圈子里有”癌症”之称——极难维护和运行
- **Turbopuffer**：在 Object Storage 上构建的向量数据库，Cursor 在实际生产中使用
- **分析引擎**：Snowflake、Databricks 等利用列式存储和向量化运算的分析系统

Asif 的数据库哲学

”扩展数据库的最好方式就是**不要有数据库**（The best way to scale a database is to just not have a database）。”这句话的深层含义是：尽量将数据存储下推到极其可靠的 Object Storage 层，用计算层做数据组织和查询，避免运维传统数据库的巨大开销。

6.3 本章小结

Object Storage 正在成为新一代数据库的基础存储层。它消除了传统数据库存储层的复杂性，提供了极高的可靠性和可扩展性。WarpStream 和 Turbopuffer 是这一趋势的代表。

7 模型推理与多 Provider 策略

7.1 全球 GPU 部署

Cursor 的自研模型推理集群分布在全球多个地点：

区域	位置
美国东海岸	Virginia
美国西部	Phoenix
欧洲	London (曾尝试 Frankfurt, 不稳定后放弃)
亚洲	Tokyo

表 1: Cursor GPU 集群全球分布

全球化部署的目标是确保无论用户身在何处（如日本），都能获得足够低延迟的 Autocomplete 体验。

7.2 Frontier Model 供应商管理

Asif 坦言，与第三方模型供应商的合作是 Cursor 基础设施中**最混乱的部分之一**：

- 多个供应商经常宕机，所有供应商的历史可靠性都很差
- 需要在多个供应商之间做负载均衡和故障转移
- Rate limit 谈判是持续的”战争”

推理供应链的连锁反应

当 Anthropic 发布 Claude 3.5 Sonnet 时，Cursor 的联合创始人 Aman 在发布前夜紧急联系课程主持人 Andre，请求帮忙联系 Anthropic 提升配额。实际的调用链条是：

Cursor → Anthropic → Google (TPU) → Borg 团队（硬件上架）

每一层都在紧急申请更多资源，整个 AI 推理供应链处于持续的”压力锅”状态。

Cursor 的应对策略包括：

- 同时与多个供应商保持合作（AWS Bedrock、Google Cloud、Anthropic 直连等）
- 为自研模型也采用多云策略（multi-cloud）
- 根据各供应商的实时可用性动态分配用户流量

推理服务的成熟度

Asif 指出，AI 推理服务整体还**不够成熟**。供应商的 caching 机制不完善，冷启动问题普遍存在。一个未命名的供应商在 Cursor 扩容时，在 30–40 million tokens/min 就开始崩溃，而 Cursor 的需求是 100 million tokens/min 以上。

7.3 本章小结

Cursor 在全球部署 GPU 集群以降低 Autocomplete 延迟，同时与多个 Frontier Model 供应商合作以提高可靠性。推理供应链的每一层都面临容量瓶颈，Rate limit 谈判是日常工作的重要组成部分。

8 安全、定价与反滥用

8.1 代码安全

面对学生关于代码安全的提问，Asif 介绍了 Cursor 在安全方面的设计：

客户端加密的向量数据库

Cursor 对用户代码的 embedding 向量进行加密存储：

- 加密密钥存储在**用户设备上**，而非服务端
- 即使攻击者获取了向量数据库的访问权限，也无法解密向量
- 虽然 Asif 个人认为从向量反推代码的可能性极低（99.99%），但由于这不是一个可证明的事实，额外的加密层提供了更强的安全保障

8.2 定价策略

Cursor 的月费为 \$20，Asif 透露了维持这个价格的工程挑战：

- 每次击键都涉及约 1000 亿次浮点运算，成本不低
- 团队投入大量工程精力优化系统，以维持 \$20 的定价
- 如果不做这些优化，更简单的做法是直接涨价
- 对于行业用户来说 \$20 微不足道，但 Cursor 希望保持对个人开发者的可及性

8.3 反滥用工程

创意无限的滥用者

Asif 分享了一个近期案例：某人创建了数十万个 Hotmail 账号（选择相对稳定的邮件域名，而非可疑的一次性域名），从约 10,000 个不同的 IP 地址发送请求，分散在约 100,000 个用户账号中。目的是利用免费试用获取大量模型推理 tokens，可能用于运行自己的服务。应对这类滥用需要深入分析流量模式来识别和封锁。

Asif 半开玩笑地向学生发出“负责任披露”邀请：如果能找到获取免费 tokens 的 bug 并报告给 Cursor，将获得免费订阅。

8.4 本章小结

Cursor 通过客户端加密保护用户代码安全，投入大量工程资源维持 \$20 定价，并持续对抗利用免费试用的滥用行为。安全和反滥用是持续的工程战场。

9 AI 对软件工程的影响

9.1 竞争与差异化

面对”市场拥挤”的质疑（GitHub Copilot、Codeium 等），Asif 的回答直截了当：

- 竞争激烈**只是事后看起来如此**，在 Cursor 创立时市场格局完全不同
- GitHub Copilot 虽然有 Microsoft + OpenAI 的”梦之队”背书，但产品在一年半内几乎没有变化
- AI 编程的天花板极高——可以上升到自动化大量工程工作——而 Copilot 只停留在 Autocomplete 层面
- 真正的竞争壁垒在于**产品创新的速度**，而非既有资源

9.2 CS 教育是否过时？

这个有些挑衅的问题引发了有趣的讨论。Asif 的观点是：

AI 让工程师更有创造力，而非取代

- 无聊的、重复性的编码工作会被自动化（如给函数添加参数）
- 但这释放了工程师去做**更复杂、更有创造性的**系统设计
- Cursor 自身就是一个例子：小团队借助 AI 辅助，覆盖了远超其人数应能处理的基础设施范围
- 类比足球运动员：移动腿部是必要的，但绝非最有意义的部分

Asif 还分享了个人经历：他在 MIT 的分布式系统课程至今仍对工作有帮助，但大部分运维技能只能在实践中学到。

9.3 IDE 的未来

对于”IDE 是否会过时”的问题，Asif 的回答极为简洁：未来仍然会有程序员使用的工具来编写代码，至于那个工具叫不叫 IDE，只是定义问题。

9.4 AI 辅助的事故响应

Asif 提到了一个令人兴奋的方向：用 AI 模型进行代码审查（code review），自动捕捉可能导致重大事故的 bug。他提到的一个具体案例：导致最近一次 Sev-0/Sev-1 级别事故的那个单行代码变更，如果当时运行了即将发布的 Bug Bot，模型**能够检测到这个 bug**。

9.5 本章小结

AI 不会取代软件工程师，而是让工程师能够处理更复杂的系统。IDE 的形态会演变，但程序员的工具需求永远存在。AI 辅助的代码审查可能成为防止重大事故的有力工具。

10 总结与延伸

本次讲座从 Cursor CTO 的第一视角展示了一个快速增长的 AI 编程产品背后的基础设施全景。以下是核心要点：

1. **规模带来全新的工程挑战**：百倍增长不是线性加服务器的问题，而是涉及架构选型、故障隔离、冷启动等系统性问题。
2. **数据库选型的实用主义**：从 YugaByte 到 PostgreSQL (RDS) 再到 Object Storage 的演进表明，应该从最简单可靠的方案开始，而不是追求理论上的完美。
3. **PostgreSQL 的 MVCC 特性需要特别注意**：对于高频更新的工作负载，VACUUM 管理是关键。Update = Delete + Insert 这个特性在面试中是知识点，在生产中是生死线。
4. **Object Storage 是新的数据库基石**：S3/R2 等 Object Storage 提供了无与伦比的可靠性和可扩展性，越来越多的数据库和流处理系统在其上构建。
5. **事故响应需要人才和文化**：并行执行多个恢复方案、关键时刻有能“苏醒”的人才、以及快速决策的文化，比完美的流程更重要。
6. **AI 推理供应链尚不成熟**：所有模型供应商的可靠性都不理想，多 Provider 策略和动态负载均衡是必要的生存手段。
7. **监控的粒度决定了发现问题的速度**：全局错误率可能掩盖局部灾难，需要在缓存命中率、特定文件类型的处理状态等细粒度维度建立监控。
8. **安全是持续的工程投入**：客户端加密、流量分析、反滥用系统都需要持续迭代。
9. **AI 辅助工程是正反馈循环**：Cursor 自身使用 AI 来构建更好的 AI 编程工具，这个循环正在加速。

10.1 拓展阅读

- **Google Spanner**: Spanner: Google's Globally-Distributed Database (2012)
- **PostgreSQL MVCC**: PostgreSQL 官方文档 — Multiversion Concurrency Control
- **WarpStream**: Kafka 兼容的 Blob Storage 原生流处理系统
- **Turbopuffer**: 基于 Object Storage 的向量数据库
- **FoundationDB**: Apple 维护的分布式事务 KV 存储
- **Merkle Tree**: Wikipedia — Merkle Tree 原理与应用
- **Cursor**: AI-First 代码编辑器